



University of Asia Pacific

Artificial Intelligence and Expert Systems Lab

Course code: **CSE 404**

Job Recommender System

A Knowledge base prolog project

Submitted to

Nahida Marzan

Lecturer, Department of CSE,
University of Asia Pacific

Submitted By

Name : Mohotasir Al Mamun

Registration ID : 22101036

Section : A

[source code](#)

Introduction

A Job Recommender System is an intelligent application that suggests relevant job opportunities to users based on their skills, experience, preferred location, and areas of interest. This system leverages Knowledge-Based System concepts and is implemented in Prolog, using facts, rules, and queries to match users with the most suitable jobs.

Problem Statement

Finding a job that matches a candidate's profile is time-consuming and often inefficient. Traditional job portals present too many irrelevant job postings. There is a need for an intelligent recommendation system that can filter jobs based on a candidate's specific skills, experience, and preferences, and additionally suggest learning resources for missing skills.

Objectives

- To create a **Knowledge Base** of users, jobs, companies and courses.
- To design rules in Prolog for matching user profiles with job requirements.
- To recommend suitable jobs to users based on predefined conditions.
- To suggest relevant courses when users lack certain job-required skills.

Tools & Technologies

- **Language: Prolog (SWI-Prolog)** – A logic programming language well-suited for building rule-based systems. SWI-Prolog provides rich built-in predicates and an efficient reasoning engine.
- **Paradigm: Logic Programming** – Uses facts and rules to define relationships and applies inference to answer queries and make decisions.
- **IDE: SWI-Prolog Editor** – Offers an environment for writing, running, and debugging Prolog code with syntax highlighting and query console.
- **Database: Built-in Prolog Facts** – Stores all user, job, and course data directly within .pl files, eliminating the need for an external database.

- **Diagram Tool:** Lucidchart is used for creating ER diagrams illustrating relationships between different entities.

System Overview

The system stores facts about:

1. **user(ID, Name, Skills, Experience, PreferredLocation, InterestFields, BlacklistedCompanies)**

Stores information about each candidate, including their unique ID, name, list of skills, total years of experience, preferred work location, fields of interest, and companies they do not wish to work for.

2. **job(ID, Title, RequiredSkills, MinimumExperience, Location, Field, Company, Salary)**

Represents a job posting with a unique ID, job title, skills required, minimum experience needed, location of work, job field or domain, hiring company, and offered salary.

3. **course(CourseName, SkillTaught)**

Lists training or learning courses, showing the course name and the specific skill it teaches.

4. **company(Name, Field, Size, Type)**

Stores details about companies, including company name, field of operation, company size (e.g., small, medium, large), and type (e.g., startup, corporate, NGO).

5. **preference(UserID, Type)**

Specifies a user's job preference type, such as *onsite*, *remote*, or *hybrid*, linked to their user ID.

6. **availability(UserID, Date)**

Indicates when a user is available for starting a job, based on their unique ID and a specific date.

7. project(Name, UserID)

Represents a project done by a user, storing the project's name and the user ID of the person who worked on it.

The rules based on:

Admin Functions (Dynamic)

- **add_job/8** → Inserts a new job posting into the database with all required details.
- **remove_job/1** → Deletes a job from the database using its unique ID.
- **remove_job_by_title/1** → Deletes a job from the database using its title.
- **add_company/4** → Registers a new company in the system.
- **remove_company/1** → Deletes a company from the database using its name.
- **add_user/7** → Adds a new user to the database after checking for duplicate IDs.

Job Recommendation

- **has_all_skills/2** → Checks if a user has all the required skills for a job.
- **eligible/2** → Determines if a user meets skill, experience, and blacklist criteria for a job.
- **recommend_jobs/2** → Returns a list of jobs a user qualifies for.
- **job_details/8** → Retrieves full details of a specific job by ID.
- **show_jobs_details/1** → Displays details for a given list of jobs.
- **recommend_and_show/1** → Shows job recommendations for a specific user along with details.

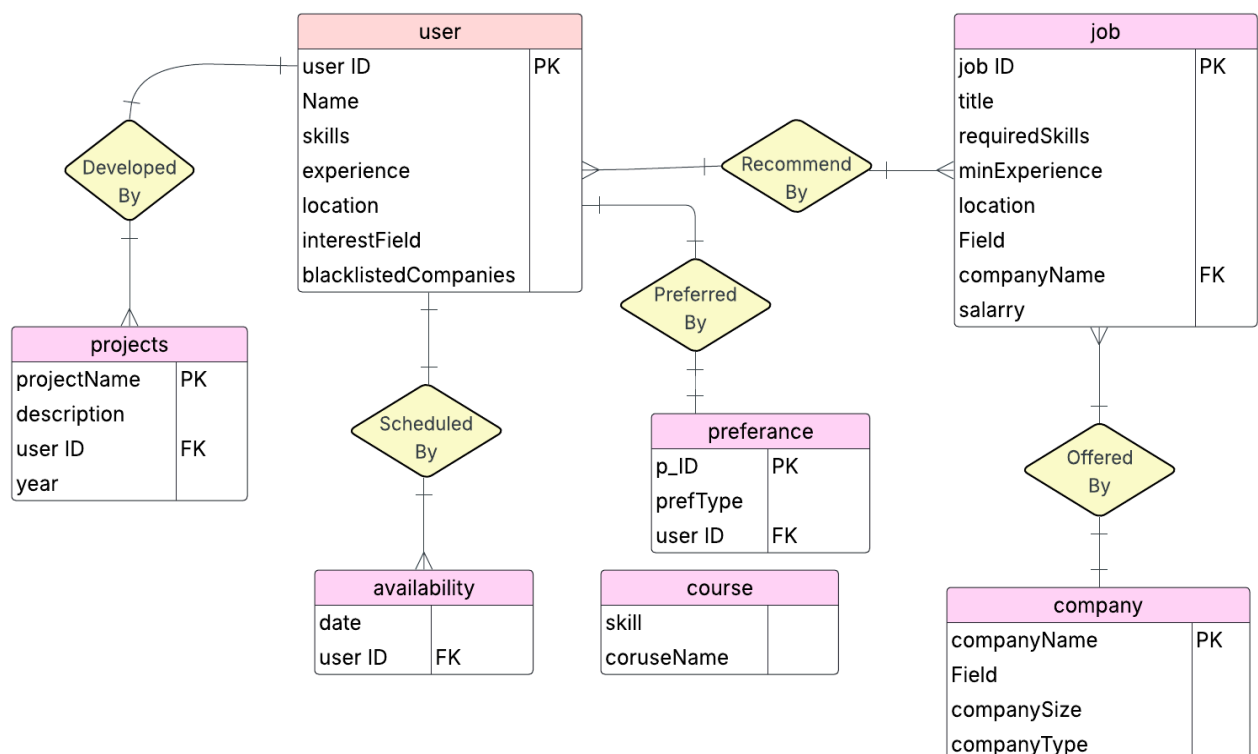
Course Suggestions

- **missing_skill/3** → Finds a missing skill for a given user and job.
- **missing_skills_list/3** → Returns all missing skills for a user to qualify for a job.
- **suggest_courses_for_job_list/3** → Pairs missing skills with available courses.
- **suggest_courses_for_job/2** → Displays course recommendations for a specific job for a user.

Company Insights

- **count_candidates_for_job/2** → Counts the number of eligible candidates for a job.
- **count_remote_perf/1** → Counts how many users prefer remote jobs.
- **count_all_jobs/1** → Returns the total number of jobs in the database.

Entity Relationship Diagram



Entities (with Attributes)

1. **User** (userID (PK), Name, Skills, Experience, Location, InterestField, BlacklistedCompanies)
 - Stores details of job seekers including their skills, work experience, preferred location, areas of interest, and companies they do not want to work for.

2. **Job** (jobID (PK), Title, RequiredSkills, MinExperience, Location, Field, CompanyName (FK), Salary)
 - Contains job postings with details such as required skills, minimum experience, job location, and associated company.
3. **Company** (companyName (PK), Field, CompanySize, CompanyType)
 - Stores details of companies offering jobs, including their industry field, size, and type.
4. **Preference** (prefType, userID (FK))
 - Records each user's work preference (e.g., remote, onsite).
5. **Course** (Skill, CourseName)
 - Contains courses available to help users acquire specific skills.
6. **Projects** (projectName (PK), Description, userID (FK), Year)
 - Stores details of projects developed by users, including descriptions and year of completion.
7. **Availability** (Date, userID (FK))
 - Records the dates when a user is available for interviews or job opportunities.

Relationships (with Entities)

1. **Developed By** (Projects – User)
 - A project is developed by a user. A user can develop multiple projects. (1:N)
2. **Scheduled By** (Availability – User)
 - Availability is scheduled by a user. A user can have multiple availability records. (1:N)
3. **Preferred By** (Preference – User)
 - A preference is chosen by a user. A user has one preference. (1:1)
4. **Offered By** (Job – Company)
 - A job is offered by a company. A company can offer multiple jobs. (1:N)
5. **Recommend By** (User – Job)
 - A job can be recommended to a user. A user can receive multiple job recommendations. (M:N)

Sample Input & Output

Query -1 :

The **show_all_jobs** rule displays all job listings from the knowledge base by retrieving each job fact and printing its details.

```
show_all_jobs :-  
    job(ID, Title, Skills, Exp, Location, Field, Company, Salary),  
    write('Job ID: '), write(ID), nl,  
    write('Title: '), write(Title), nl,  
    write('Skills Required: '), write(Skills), nl,  
    write('Experience: '), write(Exp), write(' years'), nl,  
    write('Location: '), write(Location), nl,  
    write('Field: '), write(Field), nl,  
    write('Company: '), write(Company), nl,  
    write('Salary: '), write(Salary), write(' BDT'), nl,  
    write('-----'), nl,  
    fail.
```

```
-----  
Job ID: 12  
Title: Product Manager  
Skills Required: [communication,management,roadmap]  
Experience: 4 years  
Location: chittagong  
Field: management  
Company: nexgen  
Salary: 80000 BDT  
-----  
Job ID: 13  
Title: DevOps Engineer  
Skills Required: [docker,kubernetes,git]  
Experience: 3 years  
Location: khulna  
Field: devops  
Company: cloudsmith  
Salary: 70000 BDT  
-----
```

Query - 2 :

The **show_all_company** rule displays all company records stored as company facts in the database. For each company, it prints the company name, size, type, and field of operation in a formatted manner

```

show_all_company :-
    company(Name, Field, Size, Type),
    write('Name: '), write(Name), nl,
    write('Company size: '), write(Size), nl,
    write('Type: '), write(Type), nl,
    write('Field: '), write(Field), nl,
    write('-----'), nl,
    fail.

```

```

?- show_all_company.
Correct to: "show_all_company"?
Please answer 'y' or 'n'? yes
Name: technova
Company size: large
Type: mnc
Field: [software,backend]
-----
Name: webverse
Company size: medium
Type: tech
Field: [web,frontend]
-----
Name: datacore
Company size: large
Type: corporate
Field: [data,analytics]
-----

```

Admin tasks: (- dynamic job/8, user/6, preference/2, company/4, course/2.)

Query - 3 :

The **add_job** rule adds a new job entry to the database by asserting a job fact with the given details (ID, title, skills, experience, location, field, company, and salary). After adding the job, it prints a confirmation message saying "job added successfully".

```

add_job(ID, Title, Skills, Exp, Loc, Field, Company, Salary) :-
    assertz(job(ID, Title, Skills, Exp, Loc, Field, Company, Salary)),
    write('job added successfully !!').

```

Output :

```

?- add_job(33,' JS developer' , [js,react,html],3,Dhaka, software,Techno,40000).
job added successfully !!
true.

```


Query - 4 :

These two Prolog rules are used to remove job entries from the database: **remove_job(ID)** removes a job based on its unique ID by retracting a `job` fact that matches the given ID. **remove_job_by_title(Title)** removes a job based on its title by retracting a `job` fact that matches the given title. Both use the `retract/1` predicate to delete the first matching fact from the knowledge base.

```
remove_job(ID) :-  
    retract(job(ID, _, _, _, _, _, _)).  
  
remove_job_by_title(Title) :-  
    retract(job(_, Title, _, _, _, _, _)).
```

Output:

```
?- remove_job(23).  
job delete successfully  
true.
```

Query - 5 :

The **add_user** rule adds a new user to the database if a user with the same ID doesn't already exist. It checks for uniqueness using `\+ user(...)`, and if the ID is new, it asserts the `user/7` fact and confirms success. If the user ID already exists, the second clause triggers, showing an error message and failing to prevent duplicate entries.

```
add_user(Id, Name, Skills, Experience, PreferredLocation, InterestFields, BlacklistedCompanies) :-  
    \+ user(Id, _, _, _, _, _, _) , % Check user doesn't already exist  
    assertz(user(Id, Name, Skills, Experience, PreferredLocation, InterestFields, BlacklistedCompanies)),  
    write('User added successfully.'). nl.  
  
add_user(Id, _, _, _, _, _, _) :-  
    user(Id, _, _, _, _, _, _),  
    write('Error: User already exists.'). nl, fail.
```

Output:

```
?- add_user(u55, adnan, [python, java, git, ml, database], 3, dhaka, [software, ml], []).
User added successfully.
true.

?- add_user(u5, adnan, [python, java, git, ml, database], 3, dhaka, [software, ml], []).
Error: User already exists.
false.
```

Complex Query : (using Recursion)

Query - 6 :

The **has_all_skills** predicate uses recursion to check if a user has every required skill for a job. The base case succeeds when no skills remain to check, and the recursive case verifies the first skill, then checks the rest. The **eligible/2** predicate applies multiple conditions—company not blacklisted, experience meeting the requirement, and all required skills present—to determine if a user qualifies for a job. Finally, **recommend_jobs** collects all jobs for which the user is eligible using **findall/3** and removes duplicates with **sort/2**, producing a clean list of recommended jobs.

```
has_all_skills([], _).
has_all_skills([H|T], UserSkills) :-
    member(H, UserSkills),
    has_all_skills(T, UserSkills).

eligible(Uid, JobID) :-
    user(Uid, _, UserSkills, UserExp, _, _, Blacklist),
    job(JobID, _, JobSkills, MinExp, _, _, Company, _),
    \+ member(Company, Blacklist),
    UserExp >= MinExp,
    has_all_skills(JobSkills, UserSkills).

recommend_jobs(Uid, UniqueJobIDs) :-
    findall(JobID, eligible(Uid, JobID), JobList),
    sort(JobList, UniqueJobIDs).
```

Output :

```
?- recommend_jobs(u1, U).
U = [1, 4, 9, 11].

?- 
```

Query - 7 :

The **count_candidates** predicate retrieves all user IDs and the job's requirements, then calls the recursive **count_candidates_list** predicate to count how many users qualify. The recursion processes one user at a time, checking if they are not blacklisted by the company, meet the minimum experience, and possess all required skills. If eligible, the count is incremented; otherwise, it remains the same. This continues until the list is exhausted, producing the total number of qualified candidates.

```
count_candidates(JobID, Count) :-
    findall(UserID, user(UserID, _, _, _, _, _), UserIDs),
    job(JobID, _, JobSkills, MinExp, _, _, Company, _),
    count_candidates_list(UserIDs, JobSkills, MinExp, Company, Count).

count_candidates_list([], _, _, _, 0).

count_candidates_list([H|T], JobSkills, MinExp, Company, Count) :-
    user(H, _, UserSkills, UserExp, _, _, Blacklist),
    count_candidates_list(T, JobSkills, MinExp, Company, RestCount),
    (
        \+ member(Company, Blacklist),
        UserExp >= MinExp,
        has_all_skills(JobSkills, UserSkills)
    ->
        Count is RestCount + 1
    ;
        Count is RestCount
    ).
```

Output :

```
?- count_candidates(11, Count).
Count = 8.

?-
```

Query - 8 :

The **count_remote_users** predicate collects all unique user IDs from the preference facts, then uses the recursive **count_remote_list** predicate to count how many have a remote preference. The recursion checks each user: if they prefer remote work, the count is increased; otherwise, it remains unchanged, continuing until the list is empty.

```

count_remote_list([], 0).

count_remote_list([H|T], Count) :-
    count_remote_list(T, RestCount),
    ( preference(H, remote) ->
        Count is RestCount + 1
    ;
        Count is RestCount
    ).

count_remote_users(Count) :-
    findall(UserID, preference(UserID, _), UserList),
    sort(UserList, UniqueUsers), % avoid duplicates if any
    count_remote_list(UniqueUsers, Count).

```

Output :

```

?- count_remote_users(Count).
Count = 11.

?-

```

Query - 9 :

It shows the company details by taking a company name . Show all data that a company has stored like company name, size, type .

```

company_details(Name) :-
    company(Name, Fields, Size, Type),
    format('~nCompany Name: ~w', [Name]),
    format('~nFields: ~w', [Fields]),
    format('~nSize: ~w employees', [Size]),
    format('~nType: ~w~n', [Type]).

```

Output :

```

?- company_details(codexlab).

Company Name: codexlab
Fields: [backend,web]
Size: medium employees
Type: startup
true.

?-

```

Query - 10

The **recommend_and_show** predicate first retrieves all unique job IDs suitable for a given user **using recommend_jobs**. It then displays detailed information for each recommended job through the recursive **show_jobs_details/1** predicate, which fetches job attributes using **job_details/8** and prints them in a formatted way. This approach combines filtering (eligibility check) with recursion for displaying results.

```
job_details(ID, Title, Skills, Exp, Loc, Field, Company, Salary) :-
    job(ID, Title, Skills, Exp, Loc, Field, Company, Salary).

show_jobs_details([]).
show_jobs_details([JobID|Rest]) :-
    job_details(JobID, Title, Skills, Exp, Loc, Field, Company, Salary),
    format('~nJob ID: ~w~nTitle: ~w~nSkills: ~w~nExperience: ~w years~nLocation: ~w~n
Field: ~w~nCompany: ~w~nSalary: ~w~n',
    [JobID, Title, Skills, Exp, Loc, Field, Company, Salary]),
    show_jobs_details(Rest).

recommend_and_show(Uid) :-
    recommend_jobs(Uid, UniqueJobIDs),
    write('Recommended Jobs for '), write(Uid), nl,
    show_jobs_details(UniqueJobIDs).
```

Output :

```
?- recommend_and_show(u1).
Recommended Jobs for u1

Job ID: 1
Title: Software Engineer
Skills: [python,java,git]
Experience: 2 years
Location: dhaka
Field: software
Company: technova
Salary: 60000

Job ID: 4
Title: ML Engineer
Skills: [python,java,ml,git]
Experience: 3 years
Location: rajshahi
Field: ml
Company: aibotics
Salary: 70000
```

Challenges

1. Skill Matching Accuracy:

Initial implementation failed to suggest correct jobs for candidates with partial skill overlap, resolved using findall/3 with filtering conditions.

2. Avoiding Infinite Recursion:

Early matching rules created circular calls between predicates, fixed by restructuring logic and adding termination conditions.

3. Preference Handling:

Representing remote and onsite preferences required additional facts (preference/2) and conditional checks in matching rules.

Conclusion

The Job Recommender System successfully demonstrates the application of logic programming to solve real-world matching problems between job seekers and job opportunities. By leveraging Prolog's powerful inference capabilities, the system analyzes user profiles, job requirements, and skill gaps to provide personalized job recommendations and suggest relevant courses for skill enhancement. The rule-based approach ensures flexibility, transparency, and easy scalability, making it possible to extend the system with additional criteria such as company preferences, location constraints, and industry trends. Overall, this project highlights how knowledge-based systems can bridge the gap between job seekers and the right career opportunities in an efficient and intelligent way.